

LegalEase

AI-Powered Legal Document Generator

Phase 6: Project Testing

Project Report

1. Purpose of This Phase

The Testing phase verifies that LegalEase functions correctly, reliably, and securely before it is documented and demonstrated. It covers multiple levels of testing, from individual components to the complete end-to-end workflow.

2. Testing Objectives

- Verify that each module performs its intended function correctly (unit level).
- Verify that modules work together correctly when integrated (integration level).
- Verify that the system as a whole meets the functional and non-functional requirements.
- Validate that AI-generated documents include all mandatory clauses for the chosen category.
- Confirm the system behaves acceptably from an end user's perspective (UAT).

3. Levels of Testing

3.1 Unit Testing

Individual functions and components — such as field extraction, clause validation, and export formatting — are tested in isolation using mock inputs to confirm correct outputs.

3.2 Integration Testing

Tests the interaction between the Input Processor, AI Generation Engine, and Clause Validator to ensure data flows correctly across modules and produces a coherent draft.

3.3 System Testing

The complete application is tested end-to-end, from login through document generation, editing, and export, on the deployed environment.

3.4 User Acceptance Testing (UAT)

A small group of representative users performs real tasks (e.g., generating a rental agreement) and provides feedback on usability, output quality, and any issues encountered.

4. Sample Test Cases

Test ID	Description	Expected Result	Status
TC-01	User registers with valid details	Account created, redirected to dashboard	Pass
TC-02	User logs in with incorrect password	Login rejected with error message	Pass

Test ID	Description	Expected Result	Status
TC-03	Select 'Rental Agreement' and enter details	Draft generated with all mandatory clauses	Pass
TC-04	Submit incomplete input (missing password)	System prompts user for missing field	Pass
TC-05	Edit generated draft and save changes	Updated content reflected in preview	Pass
TC-06	Export finalized document to PDF	PDF downloaded with correct formatting	Pass
TC-07	Export finalized document to Word	DOCX downloaded with correct formatting	Pass
TC-08	Access document history after login	Previously generated documents listed correctly	Pass
TC-09	Generate document with unsupported category	System displays 'category not supported' message	Pass
TC-10	Simulate AI service timeout	System shows retry/fallback message gracefully	Pass

5. Non-Functional Testing

Type	Focus	Observation
Performance Testing	Response time for document generation	Average generation time within acceptable range (under 10s)
Security Testing	Authentication and data protection	Passwords hashed; endpoints require valid tokens
Usability Testing	Ease of use for non-technical users	Users completed document generation without external assistance
Compatibility Testing	Cross-browser and device rendering	Verified on major browsers and common screen sizes

6. Defect Tracking and Resolution

Defects identified during testing were logged with a description, severity, and reproduction steps, then assigned to the relevant developer for resolution. Fixed issues were re-tested (regression testing) to confirm the fix did not introduce new problems before closure.

7. Tools Used for Testing

- Automated unit-testing framework appropriate to the backend language (e.g., PyTest / Jest).
- API testing tool (e.g., Postman) for verifying backend endpoints independently of the UI.
- Manual exploratory testing for the document editor and export features.
- Issue tracking board for logging and monitoring defects through to closure.

8. Test Summary

All planned test cases across unit, integration, system, and acceptance levels were executed. Critical and high-severity defects identified during the cycle were resolved and verified, resulting in a stable build ready for documentation and demonstration.

9. Test Environment

Testing was carried out on a staging environment that mirrors the production deployment, using representative sample data for users, templates, and documents so that results closely reflect real-world behavior.

10. Regression Testing Strategy

After each defect fix, the associated test case and a subset of related test cases were re-executed to confirm the fix resolved the issue without introducing new problems elsewhere in the system, particularly around the shared clause validation logic.

11. Test Summary Metrics

Metric	Result
Total test cases planned	10 (representative sample shown above; full suite covers all modules)
Test cases passed	All planned cases passed after defect resolution
Critical defects found	Resolved and verified through regression testing
Average generation response time	Within the target of under 10 seconds

12. Entry and Exit Criteria

- Entry criteria:** Development of the relevant module is complete and deployed to the staging environment; test data and test cases are prepared in advance.
- Exit criteria:** All planned test cases have been executed, all critical and high-severity defects are resolved and verified, and the build is stable across at least one full regression pass.

13. Lessons from the Testing Cycle

Testing reinforced the importance of the Clause Validator as a safety net for AI-generated content: several early drafts omitted clauses that were mandatory for a given document type until the validator logic was tightened, underscoring why automated clause checks are treated as a core requirement rather than an optional enhancement.

14. Outcome of This Phase

This phase confirms that LegalEase meets its functional and non-functional requirements with an acceptable level of quality, providing confidence to proceed to final documentation and demonstration.